

Databricks Certified Data Engineer Professional

Complete 3-Day Crash Course Study Guide

Aligned to the currently published Professional exam guide (live version dated 30 Nov 2025), verified in September 2026.

Built for an experienced data engineer who already uses Azure Databricks, Delta Lake, production jobs and SQL/PySpark. The guide therefore spends less time on beginner material and more time on exam decisions, newer platform features, streaming, Lakeflow, Unity Catalog, performance, governance, CI/CD and troubleshooting.

Exam: 59 scored multiple-choice questions | 120 minutes | no test aids | no prerequisite | certification validity 2 years.

How to use this book: Read the Day 1-3 path first. For each chapter, learn the decision rules, review the examples, then answer the chapter questions without looking at the answers. Finish with the rapid revision sheets and mock questions.

3-Day Study Plan

Day	Hours	Primary topics	Target
Day 1	6-8	Python/SQL, Spark execution, Delta Lake, table design, Liquid Clustering, ingestion	Build the platform + processing foundation
Day 2	7-8	Auto Loader, Structured Streaming, Lakeflow Spark Declarative Pipelines, Jobs, CDC, quality	Master production pipelines
Day 3	7-8	Unity Catalog, sharing/federation, security/privacy, monitoring, performance, CI/CD, debugging, modelling, mocks	Close exam-specific gaps and practise decisions

Exam objective map

The official outline has ten sections: (1) Developing Code for Data Processing using Python and SQL, (2) Data Ingestion & Acquisition, (3) Data Transformation, Cleansing, and Quality, (4) Data Sharing and Federation, (5) Monitoring and Alerting, (6) Cost & Performance Optimisation, (7) Ensuring Data Security and Compliance, (8) Data Governance, (9) Debugging and Deploying, and (10) Data Modelling.

KEY POINT: Do not study this like an Associate exam. Professional questions often give several technically possible answers and ask for the option that best satisfies reliability, cost, latency, governance, or maintainability.

1. Python, SQL and Production Code

1.1 Project structure and Asset Bundle mindset

A production Databricks project should separate reusable transformation logic from orchestration and environment configuration. Keep notebooks thin when possible: notebooks can call tested Python modules rather than containing all business logic. Databricks Asset Bundles (DABs) describe deployable resources and environment-specific targets so the same project can move through dev/test/prod.

```
project/  
  databricks.yml  
  resources/  
    jobs.yml  
    pipelines.yml  
  src/  
    transformations.py  
    quality.py  
  tests/  
    test_transformations.py  
  notebooks/  
    entrypoint.py
```

KEY POINT: Think: source code + resource configuration + environment targets + CI/CD. Avoid copying notebooks manually between workspaces.

1.2 Dependencies

Know the difference between workspace/runtime-provided libraries and project dependencies. Python packages may come from PyPI, wheels, or source archives. For reproducibility, pin versions where appropriate and deploy dependencies with the workload rather than relying on ad-hoc interactive installation.

EXAM TRAP: A library that works on an all-purpose development cluster can still fail on job/serverless compute if it was installed manually only on the development cluster.

1.3 UDF decision rule

Choice	Use when	Trade-off
Spark built-in functions	A built-in expression can perform the operation	Usually best optimized; Catalyst can reason about it
Python UDF	No suitable native expression and logic is Python-specific	Serialization overhead; less optimizer visibility
Pandas UDF	Vectorized Python logic is required	Arrow/vectorization can reduce Python boundary overhead, but still adds complexity

KEY POINT: Exam default: prefer native Spark SQL/DataFrame functions over Python UDFs when both solve the problem.

1.4 Testing

Professional-level code should be testable without manually inspecting notebook output. Important tools named in the exam guide include `assertDataFrameEqual`, `assertSchemaEqual`, `DataFrame.transform`, testing frameworks, and the built-in debugger. Unit tests isolate transformation logic; integration tests verify interactions among components, data, permissions, or deployed resources.

```
from pyspark.testing import assertDataFrameEqual, assertSchemaEqual  
  
actual = transform(source_df)  
assertSchemaEqual(actual.schema, expected.schema)  
assertDataFrameEqual(actual.orderBy("id"), expected.orderBy("id"))
```

1.5 SQL/PySpark patterns you must recognise

```
# Window
from pyspark.sql import functions as F, Window
w = Window.partitionBy("customer_id").orderBy(F.col("event_ts").desc())
latest = df.withColumn("rn", F.row_number().over(w)).filter("rn = 1")

# Broadcast small dimension
fact.join(F.broadcast(dim), "product_id", "left")

# Aggregate
df.groupBy("store_id").agg(F.sum("sales").alias("sales"))
```

For joins, first ask: size of each side, key distribution, required join semantics, and whether shuffle can be avoided. For windows, understand partition keys, ordering, and frame definitions. For aggregations, watch cardinality and skew.

2. Spark Execution and Performance

2.1 Mental model

A Spark action creates a job. Jobs are divided into stages, typically at shuffle boundaries. Stages contain tasks, normally one task per partition. Narrow transformations can be pipelined without redistributing data; wide transformations require data movement across executors.

Concept	Meaning	Examples
Narrow transformation	Output partition depends on a small number of input partitions	filter, map, many projections
Wide transformation	Data must be redistributed	groupBy, distinct, many joins, repartition
Shuffle	Network/disk redistribution by key or partition	expensive; can expose skew
Stage	Group of tasks that can execute without another shuffle boundary	inspect in Spark UI

2.2 Repartition vs coalesce

repartition(n) performs a shuffle and can increase or decrease partitions. Use it when you need redistribution or more balanced output. **coalesce(n)** usually reduces partitions with less data movement; it is useful when the current partitioning is already acceptable and you simply need fewer partitions.

EXAM TRAP: Do not assume coalesce gives evenly sized partitions. If the data is badly skewed, repartition may be necessary.

2.3 Broadcast joins

If one side of a join is sufficiently small, broadcasting it can eliminate a large shuffle. This is often ideal for a large fact table joined to a small dimension. It is not appropriate if the broadcast relation is too large for executor memory.

2.4 AQE

Adaptive Query Execution uses runtime statistics to improve the physical plan. It can coalesce shuffle partitions, change some join strategies, and help with skew. AQE is useful because compile-time estimates can be wrong, but it does not eliminate the need for good data modelling and table layout.

2.5 Skew

Skew occurs when a small number of keys own a disproportionate amount of data, causing a few tasks to run much longer than the rest. Diagnose it in the Spark UI/query profile by comparing task input size and duration. Remedies include AQE skew handling, salting in special cases, pre-aggregation, filtering earlier, or redesigning the join.

2.6 Cache

Cache only when reuse justifies memory pressure. Caching a huge DataFrame used once usually makes performance worse. Delta/Databricks optimizations and efficient scans can make manual caching unnecessary.

KEY POINT: Performance question sequence: scan less data -> reduce shuffle -> choose better join -> improve partition/file layout -> right-size compute -> only then consider caching or special tricks.

3. Delta Lake - Advanced Exam Core

3.1 Transaction log and ACID

Delta Lake adds a transaction log over data files. Writers commit atomic versions; readers get a consistent snapshot. This supports ACID transactions, schema enforcement, time travel and reliable batch/streaming interoperability.

3.2 MERGE

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

MERGE is appropriate for upserts and CDC-style application when you must update existing keys and insert new keys. Design the match condition carefully; duplicate source keys can make the operation ambiguous or produce unintended results depending on semantics.

3.3 Schema enforcement vs evolution

Schema enforcement prevents incompatible writes. Schema evolution intentionally permits compatible schema changes. In ingestion pipelines, separate accidental drift from approved evolution. Auto Loader can track evolving schemas; rescued data can preserve unexpected fields rather than silently losing them.

3.4 Time travel and VACUUM

Time travel reads an older table version or timestamp while the underlying files remain available. VACUUM removes files no longer needed by the current table state after the retention policy. Therefore aggressive cleanup can remove files required for older time travel.

EXAM TRAP: VACUUM is not a query-performance command. OPTIMIZE addresses file layout/compaction; VACUUM addresses obsolete-file cleanup.

3.5 OPTIMIZE, data skipping and file pruning

Small files increase metadata and scheduling overhead. Compaction creates larger files. Data skipping uses file-level statistics to avoid reading files that cannot match predicates. File pruning avoids irrelevant partitions/files. Good clustering/layout increases the effectiveness of skipping.

3.6 Deletion vectors

Deletion vectors allow some row-level changes to be represented without immediately rewriting every affected data file. This can reduce write amplification for DELETE/UPDATE/MERGE workloads. Physical file cleanup/rewrite can happen later through maintenance.

3.7 Change Data Feed

Change Data Feed (CDF) exposes row-level changes between table versions, including inserts, updates and deletes. It is useful for incremental downstream processing when consumers need changes rather than rescanning the full table.

```
SELECT *
FROM table_changes('catalog.schema.orders', 120, 125);
```

3.8 Clone

A shallow clone primarily references source data files rather than copying all data immediately; a deep clone copies data. Understand lifecycle implications: shallow clones retain dependency on source files, while deep clones consume more storage but are more independent.

4. Table Design: Partitioning, Z-ORDER and Liquid Clustering

Technique	Best mental model	Strength	Main caution
Partitioning	Physically organize by a low/moderate-cardinality column	Effective pruning when filters align	Too many partitions/small files if cardinality is high
Z-ORDER	Co-locate related values for skipping	Useful for selective predicates on chosen columns	Requires maintenance and is less flexible than liquid clustering
Liquid Clustering	Clustering keys without rigid partition layout	Flexible layout; easier evolution; reduces manual partition decisions	Choose keys based on access patterns; not a reason to ignore query design

The official Professional outline explicitly expects you to identify benefits of Liquid Clustering over partitioning and Z-ORDER. For modern managed Delta tables, liquid clustering is often the preferred strategy when access patterns evolve or rigid partitioning would be difficult.

Decision rules

- Do not partition by a near-unique key such as `transaction_id` or `user_id` on a very large table.
- A date column can be a sensible partition key when queries consistently filter by date and each partition remains reasonably large.
- Choose clustering keys that frequently appear in selective filters or joins, not columns chosen only because they are important business fields.
- Data layout cannot compensate for `SELECT *` scans with no selective predicate.

Dimensional modelling

A fact table stores measurable business events at a declared grain. Dimension tables provide descriptive context. Surrogate keys decouple warehouse identity from volatile source keys. For analytical workloads, star schemas often simplify joins and BI consumption. Define grain before selecting dimensions and measures.

KEY POINT: Exam modelling questions often become easy once you state the grain first: one row per what?

5. Ingestion and Auto Loader

5.1 Formats and sources

The exam outline calls out Delta, Parquet, ORC, Avro, JSON, CSV, XML, text and binary formats, with sources including cloud storage and message buses. The key skill is selecting a reliable incremental ingestion pattern rather than memorising file extensions.

5.2 Auto Loader

Auto Loader incrementally discovers and processes new files in cloud object storage using Structured Streaming semantics. It is designed for scalable file ingestion and maintains state so already processed files are not repeatedly treated as new input.

```
df = (spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "json")
      .option("cloudFiles.schemaLocation", schema_path)
      .load(source_path))

(df.writeStream
  .option("checkpointLocation", checkpoint_path)
  .trigger(availableNow=True)
  .toTable("main.bronze.events"))
```

5.3 Schema location vs checkpoint

Schema tracking and streaming checkpointing solve different problems. Schema information supports inference/evolution. The checkpoint tracks streaming progress/state. In current Unity Catalog guidance, checkpoint and schema information should be stored in UC-managed storage locations, and should not be nested under the table directory.

5.4 Rescued data

When unexpected or incompatible fields arrive, rescued data can preserve them for investigation rather than dropping them. This is useful when producers evolve independently and the ingestion layer must remain robust.

5.5 AvailableNow

AvailableNow provides incremental processing semantics while allowing a workload to process currently available data and then stop. It is useful for scheduled incremental ingestion where continuously running compute is unnecessary.

KEY POINT: If the business needs periodic file ingestion, do not automatically choose a continuously running stream. Compare latency requirement against compute cost.

6. Structured Streaming

6.1 Micro-batch model

Structured Streaming represents an unbounded input as an incrementally updated table. Most workloads execute as micro-batches. A trigger controls when processing is attempted; it does not guarantee completion within that interval.

EXAM TRAP: Reducing a trigger interval does not magically make a 30-second batch finish in 5 seconds. You must address throughput, skew, state, source rate or compute if processing time exceeds the SLA.

6.2 Checkpoints

A checkpoint persists progress and state needed to resume a streaming query. Treat it as part of the application's durable state. Arbitrarily deleting or reusing checkpoints can change processing semantics.

6.3 Watermarks and late data

Stateful event-time operations may need to retain state for records that could arrive late. A watermark gives the engine a bound for how late data is expected, allowing old state to be cleaned up. The trade-off is correctness for data arriving later than the accepted lateness threshold.

6.4 Stateful operations

Aggregations, deduplication and stream-stream joins can maintain state. State can grow dramatically if keys are high-cardinality or lateness is unbounded. Production design must control state size.

6.5 Stream-static vs stream-stream joins

A stream joined with a static dimension is conceptually simpler because only the stream advances. Stream-stream joins require reasoning about event time, state on both sides, and watermarks/constraints to bound state.

6.6 Exactly-once reasoning

Do not reduce exactly-once to a slogan. End-to-end semantics depend on source, checkpointing, transformations and sink. Delta sinks integrate well with Structured Streaming, but custom side effects inside `foreachBatch` must still be designed for retries/idempotency.

7. Lakeflow Spark Declarative Pipelines

Lakeflow Spark Declarative Pipelines is the current Databricks pipeline framework named in the Professional guide. It provides declarative batch/streaming pipeline definitions, managed dependencies, data-quality expectations, event logs and production orchestration features.

7.1 Streaming tables vs materialized views

Object	Think of it as	Good fit
Streaming table	Incrementally maintained table over streaming/incremental input	Append/incremental ingestion and streaming transformations
Materialized view	Persisted result of a declarative query refreshed by the platform	Derived/aggregated data where declarative refresh is desirable

The exam explicitly expects comparison of streaming tables and materialized views. Base the choice on source semantics and desired refresh behaviour, not simply 'streaming is faster'.

7.2 Expectations

Expectations express data-quality rules. Depending on the desired policy, invalid rows can be retained and measured, dropped, or cause the pipeline to fail. Pick the action based on business criticality: quarantine/drop for recoverable bad records; fail when continuing would publish unsafe data.

7.3 Quarantine pattern

A robust quarantine design separates valid and invalid records while retaining enough context to remediate rejected data. Do not merely discard malformed records if auditability is required.

7.4 APPLY CHANGES / CDC

The guide explicitly calls out APPLY CHANGES APIs for simplifying CDC. Conceptually, the framework applies ordered source changes to a target, supporting common CDC/SCD patterns while reducing custom MERGE orchestration.

7.5 Control flow

The current guide includes pipeline components using control flow such as if/else and iteration. The design goal is still declarative and maintainable: use configuration-driven generation where multiple similar datasets need consistent processing.

7.6 Event log

Pipeline event logs are a primary observability and debugging source for pipeline updates, flow progress, data-quality metrics and failures. When a Lakeflow pipeline fails, event logs plus Spark UI are core diagnostic tools.

8. Lakeflow Jobs and Orchestration

8.1 DAG semantics

A multi-task job forms a dependency graph. Successful upstream tasks do not automatically roll back because a later independent task fails. Design individual writes to be transactional/idempotent and use task dependencies to express ordering.

8.2 Job compute

Option	Use when	Reason
Job compute / new job cluster	Scheduled production workload	Isolation, reproducibility, lifecycle tied to job
Serverless compute	Supported workload where fast startup and managed operations are valuable	Less infrastructure management; elastic
Interactive/all-purpose compute	Exploration and collaborative development	Convenient for humans, generally not default for scheduled production

8.3 Retries and repair runs

Retries rerun failed work according to task/job configuration. A repair run lets you rerun failed/skipped portions of a previous run rather than repeating everything. Parameter overrides can be useful when correcting a failed production run.

KEY POINT: If A succeeded, B succeeded, and C failed, a repair run should normally focus on the failed/downstream work rather than recomputing already successful expensive tasks.

8.4 Parameters

Parameterize environment, date ranges, paths and operational controls rather than hard-coding them. Job parameters and task values can coordinate reusable workflows. Keep secrets in secret-management mechanisms rather than parameters visible in logs/UI.

8.5 Notifications

Lakeflow Jobs UI and Jobs API can configure notifications for job status and performance issues. Use notifications as part of operational readiness, not as a substitute for structured observability.

9. Unity Catalog, Sharing and Federation

9.1 Namespace and permissions

Unity Catalog provides centralized governance across catalogs, schemas and securable objects. Permissions inherit through the hierarchy according to the UC model. Common reasoning starts with `USE CATALOG / USE SCHEMA` plus object-specific privileges.

9.2 Managed vs external

Managed table	External table
Databricks/UC manages the table and underlying data lifecycle.	UC governs the table, but data is stored at an explicitly managed external location.
Lower operational burden is a key benefit called out by the exam guide.	Useful when storage location/lifecycle must remain externally controlled or shared with non-Databricks clients.

9.3 External locations and storage credentials

Storage credentials represent authentication to cloud storage; external locations associate governed cloud paths with credentials. Prefer UC-governed access rather than embedding cloud secrets directly in notebooks.

9.4 Delta Sharing

Delta Sharing provides live data sharing without requiring the recipient to receive copied extracts as the primary pattern. Databricks-to-Databricks (D2D) sharing is optimized for Databricks recipients; the open protocol (D2O) supports external platforms/clients.

9.5 Lakehouse Federation

Lakehouse Federation allows governed query access to supported external systems without first ingesting all data into Delta. It can be useful for discovery, low-copy integration and selective access. For heavy repeated transformations, ingestion into the lakehouse may still be more efficient.

EXAM TRAP: Federation and ingestion solve different problems. 'No copy' is not automatically best if the workload repeatedly scans large remote datasets.

10. Security, Privacy and Compliance

10.1 Least privilege

Grant only the permissions required for the workload. Secure both data objects and workspace resources. Avoid broad ownership or ALL PRIVILEGES grants merely to fix access errors.

10.2 Row filters and column masks

Row filters restrict which rows a principal can see; column masks transform or redact sensitive column values. These controls are preferable to proliferating many physical copies of the same dataset solely for access control.

10.3 Anonymization and pseudonymization

Technique	Idea	Reversibility / implication
Hashing	Transform value through hash function	Usually one-way in design, but weak domains can be guessed; salt/key choices matter
Tokenization	Replace sensitive value with token	Can be reversible through controlled token vault/service
Suppression	Remove or hide value	Reduces utility but strongly limits exposure
Generalisation	Reduce precision/detail	Example: exact age -> age band

10.4 PII pipeline

A compliant pipeline should identify sensitive fields, apply required masking/tokenization before broad consumption, preserve auditability, and enforce access policies. For streaming, apply the same controls continuously rather than assuming privacy is a batch-only concern.

10.5 Purging

Retention compliance can require deletion of subject data and/or expired records. Consider downstream copies, derived tables, CDF/streaming consumers, backups/retention constraints and audit evidence. A DELETE statement alone is not the entire governance process.

11. Monitoring, Alerting and Troubleshooting

11.1 System tables

System tables provide account/workspace observability for areas such as usage/cost, audit activity and workload monitoring. They are valuable for trend analysis and operational dashboards because they expose structured telemetry.

11.2 Query Profile vs Spark UI

Tool	Best for
Query Profile	Understanding SQL/query operators, scan volume, joins, shuffles, data skipping and bottlenecks
Spark UI	Stages/tasks, skew, shuffle read/write, executor behaviour, failed tasks and detailed Spark execution
Pipeline Event Log	Lakeflow pipeline update events, flow progress, data-quality metrics and pipeline failures
Jobs UI/API/CLI	Run state, task failures, operational automation and notifications

11.3 Diagnostic workflow

- 1. Identify whether the issue is correctness, latency, cost, or availability.
- 2. Find the failing/slow operator or task rather than tuning the whole platform blindly.
- 3. Inspect scan size, pruning/skipping, shuffle volume, skew and spill.
- 4. Check recent code/data/schema/config changes.
- 5. Remediate, then validate against the SLA and data-quality checks.

11.4 SQL Alerts

SQL Alerts can monitor query-derived conditions, including data-quality indicators. They are appropriate when a SQL result crossing a threshold should trigger notification/action.

12. Cost and Performance Optimisation

12.1 Scan less

The cheapest byte is the byte you never scan. Select only required columns, filter early where semantically safe, use data skipping/pruning, and maintain sensible file layout.

12.2 Shuffle less

Broadcast small dimensions, pre-aggregate when possible, avoid unnecessary distinct/orderBy/repartition operations, and choose join keys carefully.

12.3 Right-size compute

More workers do not fix every bottleneck. CPU-bound workloads may scale with compute; skewed single tasks may not. Memory pressure may require different worker characteristics or reduced state/shuffle. Serverless can reduce operational overhead for supported workloads.

12.4 File size

Many tiny files create metadata and scheduling overhead. Extremely large files can reduce parallelism. Delta maintenance features and optimized writes help keep file layout healthy.

12.5 Query-profile clues

Symptom	Likely investigation
Huge scan, few output rows	Predicate pushdown, data skipping, partition/clustering keys
One task much slower	Skew / uneven partitioning
Large shuffle	Join/groupBy/window strategy
Spill to disk	Memory pressure, partition sizing, aggregation/join design
Many tiny tasks/files	Small-file problem, over-partitioning

13. CI/CD, Asset Bundles, CLI and REST API

13.1 Asset Bundles

Databricks Asset Bundles package source and resource definitions for repeatable deployment. A bundle can define jobs, pipelines and environment-specific targets. CI/CD validates and deploys the same codebase with controlled configuration differences.

```
bundle:
  name: sales-pipeline

include:
  - resources/*.yaml

targets:
  dev:
    mode: development
  prod:
    mode: production
```

13.2 Git-based workflow

Use branches/pull requests/tests to control source changes. Databricks Git Folders support Git-integrated development in the workspace. Deployment to higher environments should be automated rather than based on manual notebook copying.

13.3 CLI and REST

The CLI and REST API are automation surfaces for deploying and operating Databricks resources. The exam guide explicitly includes using them for jobs/pipelines and monitoring. Know when API/CLI automation is preferable to a manual UI operation: repeatability, CI/CD, bulk operations and integration with external systems.

13.4 Secrets

Secrets should not be committed to Git or hard-coded in bundle YAML. Use secret-management mechanisms and identity-based access where possible. Databricks secret values displayed through supported secret retrieval are redacted in notebook output, but you should still avoid unnecessary exposure.

14. High-Value Decision Sheets

Scenario	Prefer	Why
New files continuously arrive in cloud storage	Auto Loader	Scalable incremental discovery + streaming state
Hourly incremental ingestion; no need for 24x7 compute	Auto Loader + AvailableNow / scheduled pipeline	Incremental semantics with bounded compute
Large fact + tiny dimension	Broadcast join	Avoid shuffle of large side
Evolving large Delta table; changing filter patterns	Liquid Clustering	Flexible layout without rigid partitions
Need downstream row-level changes	CDF	Consume changes instead of full rescans
Need governed live sharing to another platform	Delta Sharing	Share data without extract-copy workflow
Need selective query access to external DB without ingesting first	Lakehouse Federation	Governed federation
Bad records should be retained for remediation	Quarantine	Preserve evidence while keeping good path clean
Failed middle task in expensive DAG	Repair run	Avoid repeating successful upstream work
Reusable deployment across dev/test/prod	Asset Bundles + CI/CD	Repeatable environment-aware deployment

Common traps

- Trigger interval is scheduling intent, not guaranteed processing latency.
- Partitioning by high-cardinality IDs usually creates pathological layouts.
- VACUUM is not compaction.
- More partitions are not always faster; too many tiny tasks/files add overhead.
- Broadcasting a large table can cause memory pressure.
- Deleting a checkpoint is not a normal recovery technique.
- Streaming does not automatically mean lower cost than scheduled incremental processing.
- Managed tables reduce operational burden; external tables are not automatically 'more enterprise'.
- Security should be enforced through governed identities/policies, not hidden notebook variables.
- Successful tasks in a multi-task job are not globally rolled back because a later task fails.

15. Practice Questions - 80 Exam-Style Scenarios

1. A 4 TB fact table is joined daily to a 20 MB product dimension. Both are filtered first. What is the first join optimization to consider?

Answer: Broadcast the small product dimension, assuming it fits safely in executor memory. **Why:** This can avoid shuffling the multi-terabyte fact side.

2. A streaming aggregation retains state indefinitely and memory usage keeps growing. What concept should you examine first?

Answer: Event-time watermarking and whether state can be bounded. **Why:** Watermarks let Spark discard sufficiently old state for supported stateful operations.

3. A team partitions a Delta table by customer_id where there are 40 million customers. Queries become slower and create many tiny directories/files. What is the design issue?

Answer: Excessively high-cardinality partitioning. **Why:** Use a more suitable layout; liquid clustering may be a better fit for evolving access patterns.

4. A scheduled ingestion must process all files that arrived since the previous run and then terminate. Which streaming trigger pattern is a strong fit?

Answer: AvailableNow. **Why:** It keeps incremental/checkpoint semantics but does not require continuously running compute.

5. A downstream system needs only inserts, updates and deletes from a Delta table since the last processed version. What feature fits?

Answer: Change Data Feed. **Why:** CDF exposes row-level changes incrementally.

6. A pipeline must preserve malformed input for later remediation while keeping invalid rows out of curated output. What pattern fits?

Answer: Quarantine invalid records. **Why:** Do not silently discard data needed for investigation.

7. A query reads 8 TB but returns 1000 rows. What should you investigate before adding compute?

Answer: Data pruning/skipping and filter/table layout. **Why:** Reducing scan volume is usually more effective than brute-force compute.

8. A production DAG has A -> {B,C}; A and B succeed, C fails. Does Databricks automatically roll back A and B?

Answer: No. **Why:** Task writes are not one global transaction across the entire job DAG.

9. You need a repeatable deployment of jobs and pipelines to dev and prod with different settings. What is the core Databricks mechanism in the exam guide?

Answer: Databricks Asset Bundles. **Why:** Bundles define resources and targets for automated deployment.

10. A sensitive email column should show real values only to an authorized group while other users see masked values. What control fits?

Answer: Column masking / governed conditional view or mask. **Why:** Apply access-aware masking rather than making uncontrolled copies.

11. A query has one task running 20x longer than all others. What is the first Spark concept to suspect?

Answer: Data skew. **Why:** Inspect task input/shuffle metrics to confirm.

12. A developer uses a Python UDF for uppercasing a string. What is preferable?

Answer: Use the native Spark upper() function. **Why:** Built-ins are more optimizer-friendly and avoid Python serialization overhead.

13. A Delta table has thousands of tiny files after frequent writes. Which maintenance concern is primary?

Answer: Compaction / optimized file layout. **Why:** VACUUM is for obsolete files, not tiny-file compaction.

14. A team wants to query a supported external database through UC without initially copying the data. What feature fits?

Answer: Lakehouse Federation. **Why:** It provides governed federated access.

15. An external organization needs live governed access to selected lakehouse data without receiving periodic CSV extracts. What feature fits?

Answer: Delta Sharing. **Why:** It is designed for secure live sharing across platforms.

16. A job uses an interactive cluster that stays running all day but executes for 10 minutes each hour. What should you evaluate?

Answer: Job/serverless compute scheduled per workload. **Why:** This can reduce idle compute and improve production isolation.

17. A pipeline must fail immediately when a critical accounting invariant is violated. Which quality policy is appropriate?

Answer: An expectation/rule configured to fail the pipeline. **Why:** Critical invalid data should not be published.

18. Unexpected JSON fields appear occasionally and should be preserved for analysis rather than lost. What Auto Loader concept helps?

Answer: Rescued data. **Why:** It retains unexpected/incompatible fields for later handling.

19. A shallow clone's source data files are removed by source lifecycle operations. Why can this matter?

Answer: A shallow clone can depend on source data files. **Why:** A deep clone is more storage-intensive but independent of those source files.

20. A developer calls repartition(1) before every write to produce one file. Why is this risky at scale?

Answer: It collapses parallelism and can create a severe bottleneck. **Why:** Single-file output is usually an anti-pattern for large distributed datasets.

21. Which tool is best for examining stage/task skew and shuffle metrics?

Answer: Spark UI. **Why:** It exposes stages, tasks, shuffle, executor and failure details.

22. Which tool is especially useful for SQL operator-level bottlenecks and scan/skipping behaviour?

Answer: Query Profile. **Why:** It helps identify expensive operators, joins, scans and shuffles.

23. Where should you look for Lakeflow pipeline update and data-quality events?

Answer: Pipeline event log. **Why:** It is a primary observability source for declarative pipelines.

24. What is the key difference between schema tracking and a streaming checkpoint?

Answer: Schema tracking records schema inference/evolution; checkpoint records stream progress/state. **Why:** They solve different operational problems.

25. When should coalesce be preferred over repartition?

Answer: When mainly reducing partition count and a full redistribution is unnecessary. **Why:** Coalesce usually avoids a full shuffle.

26. When is repartition more appropriate than coalesce?

Answer: When data must be redistributed/balanced or partition count must increase. **Why:** Repartition performs a shuffle.

27. What does AQE use that a static plan does not have?

Answer: Runtime statistics. **Why:** It can adapt joins/partitions/skew decisions during execution.

28. A stream-static join uses a small static dimension. What optimization may still apply?

Answer: Broadcast the static dimension if appropriate. **Why:** Streaming does not prevent broadcast of a suitable static side.

29. Why can an unbounded stream-stream join be dangerous?

Answer: State can grow without bound. **Why:** Event-time constraints/watermarks are often needed to control state.

30. What is the first modelling question before building a fact table?

Answer: Define the grain. **Why:** Measures and dimensions must align to one declared row-level business event.

31. Why use surrogate keys in dimensions?

Answer: To provide stable warehouse identity independent of source-system key changes. **Why:** They support dimensional history and integration.

32. What is a primary advantage of managed UC tables highlighted by the exam?

Answer: Reduced operational/maintenance burden. **Why:** The platform manages more of the data lifecycle.

33. Why should storage credentials/external locations be preferred over cloud keys in notebooks?

Answer: Centralized governed access and reduced secret sprawl. **Why:** Identity and path governance belong in UC.

34. What is the security principle behind granting only SELECT rather than ALL PRIVILEGES?

Answer: Least privilege. **Why:** Grant only what the workload needs.

35. What does generalisation do to PII?

Answer: Reduces precision/detail, such as exact age to age band. **Why:** It lowers re-identification risk while retaining some analytical utility.

36. Why is tokenization different from hashing?

Answer: Tokens can be mapped back through a controlled service; hashes are typically used as one-way transforms. **Why:** The operational and security models differ.

37. What is a repair run designed to avoid?

Answer: Re-running already successful expensive tasks unnecessarily. **Why:** It targets failed/skipped downstream work.

38. What should job parameters not contain?

Answer: Plaintext secrets. **Why:** Use secret management/identity instead.

39. Why are unit tests valuable for DataFrame.transform-based code?

Answer: Transformation functions can be tested independently with known inputs/outputs. **Why:** This reduces reliance on manual notebook validation.

40. What should happen when a dependency exists only on a developer's interactive cluster?

Answer: Package and declare the dependency for the production workload. **Why:** Manual cluster state is not reproducible.

41. What is the difference between OPTIMIZE and VACUUM?

Answer: OPTIMIZE improves file layout; VACUUM removes obsolete files after retention. **Why:** They solve different storage concerns.

42. What enables time travel in Delta?

Answer: Versioned transaction-log history plus retained underlying files. **Why:** If old files are removed, older versions may no longer be readable.

43. What does data skipping rely on?

Answer: File-level statistics/layout to avoid reading irrelevant files. **Why:** Better clustering can improve its effectiveness.

44. What is file pruning?

Answer: Avoiding files/partitions known not to satisfy filters. **Why:** It reduces I/O before row processing.

45. Why can SELECT * hurt performance?

Answer: It reads unnecessary columns and increases I/O/processing. **Why:** Project only required columns.

46. What is the risk of broadcasting a medium/large relation indiscriminately?

Answer: Executor memory pressure or failures. **Why:** Broadcast is useful only when the relation is safely small enough.

47. Why can increasing cluster size fail to fix skew?

Answer: The bottleneck may be one/few oversized partitions. **Why:** Redistribution/skew handling is required.

48. When is caching a poor choice?

Answer: When the dataset is used once or is too large relative to memory. **Why:** Cache has materialization and memory costs.

49. What is the main benefit of CDF over rescanning the whole source?

Answer: Incremental change consumption. **Why:** Downstream systems process only relevant changes.

50. What is the difference between D2D and D2O sharing?

Answer: D2D is Databricks-to-Databricks; D2O uses the open sharing protocol for external platforms. **Why:** Choose based on recipient platform.

51. When may federation be worse than ingestion?

Answer: Repeated heavy scans/transforms over remote data. **Why:** Network/source load can make local Delta ingestion more efficient.

52. What does a row filter protect?

Answer: Which rows a user/principal can access. **Why:** Column masks protect values in columns instead.

53. What does a column mask protect?

Answer: Sensitive column values based on policy/principal. **Why:** It can expose transformed/redacted values rather than raw values.

54. Why is purging more than DELETE?

Answer: Retention may involve downstream copies, derived data, streaming consumers and audit requirements. **Why:** Compliance is an end-to-end lifecycle concern.

55. Which mechanism can alert when a SQL quality metric crosses a threshold?

Answer: SQL Alerts. **Why:** They evaluate query-derived conditions.

56. Which interfaces can automate monitoring/operations of jobs and pipelines?

Answer: Databricks REST API and CLI. **Why:** They support repeatable external automation.

57. Why use Git pull requests for pipeline code?

Answer: Review, traceability and controlled promotion. **Why:** It reduces uncontrolled workspace edits.

58. Why use separate DAB targets?

Answer: To apply environment-specific settings while keeping one deployable codebase. **Why:** This supports dev/test/prod promotion.

59. Should secrets be stored in databricks.yml?

Answer: No. **Why:** Use secret management or identity-based mechanisms.

60. A pipeline needs CDC/SCD handling with less custom MERGE orchestration. What exam-listed capability should you know?

Answer: APPLY CHANGES APIs in Lakeflow pipelines. **Why:** They simplify declarative CDC processing.

61. What should determine whether to use a streaming table or materialized view?

Answer: Input semantics and desired refresh/maintenance behaviour. **Why:** Do not choose based on the word 'streaming' alone.

62. What is the benefit of declarative pipeline dependencies?

Answer: The platform can manage execution ordering and incremental refresh semantics. **Why:** You describe desired datasets/flows rather than hand-orchestrating every step.

63. Why use a quarantine table rather than simply drop invalid rows?

Answer: Auditability and remediation. **Why:** It preserves rejected records and reasons.

64. What does schema enforcement protect against?

Answer: Unintended incompatible writes. **Why:** Evolution should be intentional.

65. What does schema evolution enable?

Answer: Controlled acceptance of compatible schema changes. **Why:** It supports changing upstream data when policy permits.

66. What does a Delta transaction provide to readers during concurrent writes?

Answer: A consistent snapshot. **Why:** Readers do not see partial commits.

67. What is a common partitioning candidate compared with unique IDs?

Answer: A date column with suitable cardinality and query alignment. **Why:** It can enable effective pruning without exploding partitions.

68. Why is Liquid Clustering attractive for evolving access patterns?

Answer: Clustering keys/layout can be managed more flexibly than rigid directory partitioning. **Why:** It reduces up-front partition-design burden.

69. Does Liquid Clustering remove the need to choose useful keys?

Answer: No. **Why:** Keys should still reflect important selective access patterns.

70. What is the danger of excessive small files?

Answer: Metadata, scheduling and I/O overhead. **Why:** Compaction/optimized writes help.

71. What is the danger of too few huge partitions?

Answer: Insufficient parallelism and long-running tasks. **Why:** Balance file/partition sizing with workload scale.

72. What is a wide transformation?

Answer: One requiring redistribution across partitions, such as groupBy or many joins. **Why:** It usually introduces a shuffle boundary.

73. What is a narrow transformation?

Answer: One where output partitions can be computed from limited input partitions without redistribution. **Why:** These can often be pipelined.

74. What is a Spark stage boundary commonly associated with?

Answer: A shuffle. **Why:** Wide dependencies split stages.

75. What is a Spark task commonly associated with?

Answer: Work on one partition within a stage. **Why:** Task counts relate to partitioning.

76. What does spill indicate?

Answer: Working data exceeded available in-memory execution capacity and spilled to disk. **Why:** Investigate memory, partition size and operator design.

77. What does high shuffle read/write suggest?

Answer: Expensive redistribution from joins/aggregations/windows/repartitioning. **Why:** Reduce unnecessary shuffle or improve strategy.

78. Why filter before a large join when semantically valid?

Answer: It reduces rows participating in the join/shuffle. **Why:** Less data movement generally improves cost/performance.

79. Why pre-aggregate before a join in some pipelines?

Answer: It can reduce data volume before expensive redistribution. **Why:** Only do so when semantics permit.

80. What is idempotency in a retryable pipeline?

Answer: Repeating the operation produces the intended state without duplicate side effects. **Why:** Critical for reliable retries/foreachBatch/external writes.

16. Final 2-Hour Revision Sheet

Memorise these contrasts

A	B	Difference to remember
OPTIMIZE	VACUUM	File layout/compaction vs obsolete-file cleanup
Watermark	Checkpoint	Bound late-event state vs durable stream progress/state
CDF	MERGE	Read changes vs apply/upsert changes
Broadcast	Shuffle join	Replicate small side vs redistribute by key
repartition	coalesce	Shuffle redistribution vs mainly reduce partitions
Managed table	External table	Platform-managed lifecycle vs externally located data
Delta Sharing	Federation	Share lakehouse data vs query external systems
Row filter	Column mask	Restrict rows vs transform sensitive values
Streaming table	Materialized view	Incremental streaming-oriented table vs persisted declarative query result
Repair run	Full rerun	Repeat failed/skipped work vs rerun entire workflow

Last-minute checklist

- Can I explain why Liquid Clustering may be preferred to partitioning/Z-ORDER?
- Can I diagnose scan, shuffle, skew, spill and small-file problems?
- Can I explain checkpoint, watermark, state and AvailableNow without mixing them up?
- Can I choose between Auto Loader, Structured Streaming and Lakeflow pipelines?
- Can I explain streaming table vs materialized view and CDC/APPLY CHANGES?
- Can I reason about UC managed/external tables, permissions, row filters and masks?
- Can I choose Delta Sharing vs Lakehouse Federation?
- Can I use system tables, Query Profile, Spark UI and pipeline event logs appropriately?
- Can I explain DAB targets, Git CI/CD, CLI/REST and dependency packaging?
- Can I recover a failed job with repair runs and parameter overrides?

Exam timing

59 scored questions in 120 minutes is about 2 minutes per scored question. On the first pass, answer clear questions quickly and mark uncertain scenario questions. On the second pass, identify the requirement that actually differentiates the options: cost, latency, correctness, governance, or operational simplicity.

KEY POINT: When two answers both work technically, choose the one that best matches Databricks-managed, governed, incremental, production-ready behaviour with the least unnecessary operational complexity.

17. Source and Scope Notes

This study guide is aligned to the official Databricks Certified Data Engineer Professional exam guide whose currently published live version is dated 30 November 2025. Databricks states that the guide is updated when the exam changes and recommends checking again two weeks before the exam.

Official exam guide: Databricks Certified Data Engineer Professional - November 30, 2025.

Current Databricks documentation was also checked for Auto Loader with Unity Catalog and Lakeflow pipeline terminology. Product behaviour can evolve, so use the official exam guide as the scope authority and current Databricks documentation for implementation details.

Important: The practice questions in this book are original study questions, not leaked or reproduced live exam questions.